# Codable

# JSON

Javascript Object Notation (**JSON**) has become the de facto standard for much of the data transferred between client and server. JSON's basic types are:

- **Number**
- **String**
- **Boolean**
- **Array**
- **Object** (key-value data)
- **null**

# JSON

From PokéAPI GET Bulbasaur


```
{
    "id": "226bfcf6-05ed-4668-8522-d5df6604a11c",
    "name": "bulbasaur",
    "height": 7.45,
    "weight": 69,
    "abilities": [
        {
            "name": "overgrow",
            "is_hidden": false
        }
    ],
    "sprites": null
}
```

# JSON in Swift

- In Swift, we use a special type called `Any` when we work with **JSON**
- `Any` is a generic placeholder for **any type whatsoever**
- `AnyObject` is similar but is limited to **class instances**

# Examples

- Swift JSON Dictionary: `[String: Any]`
- Swift JSON Array: `[Any]`
- Swift Array of Dictionaries: `[[String: Any]]`

# Data

- Literally, just a **collection of bytes**
- Used by `JSONSerialization` to create **JSON** objects
- Type returned by `JSONSerialization` when invoking

```
JSONSerialization.data(withJSONObject:, options:)
```

- Used by `JSONDecoder` to **decode** an object
- Type returned by **encoding** an object with `JSONEncoder`

# JSONSerialization

- A library for parsing `Data` into **JSON** (and vice versa)

- Methods can `throw` an exception (**MUST** be marked with `try`)

# JSONSerialization

When parsing JSON **from** `Data`:

- Returns an instance of type `Any`

```
let result: Any = try JSONSerialization.jsonObject(with: usableData, options: [])
```

- Cast the `Any` result to a usable type if necessary

```
let result = (try JSONSerialization.jsonObject(with: usableData, options: [])) as! SomeType
```

- Where `SomeType` conforms to a valid **JSON** structure
  - Example:

```
typealias SomeType = [String: Any]
```

# JSONSerialization

When parsing JSON **into** `Data`:

- Object being transformed **MUST** be a valid **JSON** object
- Invoke following method:

```
JSONSerialization.data(withJSONObject:, options:)
```

- Example:

```
let data = try JSONSerialization.data(withJSONObject: jsonObject, options: [])
```

# Codable

- Great for interfacing with **JSON**
- Utilized for tasks such as:
  - Sending data over a network connection
  - Saving data to disk
  - Submitting data to APIs and services
- A combination of `Encodable` and `Decodable` `protocols`
  - Literally:

```
typealias Codable = Decodable & Encodable
```

# Codable

- Most **Swift Standard Library Types** already conform to this protocol, including:

  - **String**
  - **Int**
  - **Double**
  - **Data**
  - **Date**
  - **URL**
  - **Array**
  - **Dictionary**
  - **Optional**

- A type whose properties are `Codable` automatically conforms to `Codable` just by *declaring* conformance

- Otherwise, `init(from:)` and `encode(to:)` functions **MUST** be implemented

# Codable

```
struct Landmark: Codable {
    let name: String
    let foundingYear: Int

    // Landmark now supports the Codable methods init(from:) and encode(to:),
    // even though they aren't written as part of its declaration.
}
```

# Encodable

- `protocol` for converting an **object to** `Data`
- Can be declared on a type in place of `Codable` if decoding is not necessary
- When encoding occurs:
  - Invokes `` `func encode(to encoder: Encoder) throws` `` under the covers

# JSONEncoder

- Object that encodes instances of a data type as JSON objects

- Invoke `func encode(_ value:)` passing the value to be encoded as a parameter

# JSONEncoder

```swift
struct GroceryProduct: Codable {
    let name: String
    let points: Int
    let description: String?
}
```

```swift
let pear = GroceryProduct(name: "Pear", points: 250, description: "A ripe
pear.")
let data = try JSONEncoder().encode(pear)

/**
 Contents of `data: Data` are as follows

 {
   "name": "Pear",
   "points": 250,
   "description": "A ripe pear."
 }
*/
```

# Decodable

- `protocol` for converting an `Data` **to an object**
- Can be declared on a type in place of `Codable` if encoding is not necessary
- When decoding occurs:
  - Invokes `` `init(from decoder: Decoder) throws` `` under the covers

# JSONDecoder

- Object that decodes instances of a data type from JSON objects

- Invoke `func decode(_ type:, from data:)` which takes the following parameters:

  - **type**: The type of the value to decode from the supplied JSON object

  - **data**: The JSON object to decode

# JSONDecoder

```
struct GroceryProduct: Codable {
    var name: String
    var points: Int
    var description: String?
}

let json = """
{
    "name": "Durian",
    "points": 600,
    "description": "A fruit with a distinctive scent."
}
""".data(using: .utf8)!

let product = try JSONDecoder().decode(GroceryProduct.self,
from: json)

print(product.name) // Prints "Durian"
```

# CodingKey

- `protocol` signifying a type that can be used as a key for encoding and decoding

- `Codable`/`Encodable`/`Decodable` types can declare a special nested `enum` named `CodingKeys` that conforms to the `CodingKey` `protocol`

- When present; serves as the authoritative list of properties that must be included when encoding/ decoding

- Names of the `enum` cases should match the names of the properties

# CodingKey

- Omit properties from `CodingKeys enum` if
  - They won't be present when decoding instances
    - Certain properties shouldn't be included in an encoded representation
- Provide **alternative** keys by specifying `String` as the raw-value type for the `CodingKeys enum` and identify the `rawValue` to be used

# CodingKey

```swift
struct Landmark: Encodable {
    let name: String
    let foundingYear: Int
    let location: String
    let vantagePoints: [CLLocation]

    enum CodingKeys: String, CodingKey {
        case name = "title"
        case foundingYear = "founding_date"
        case location
    }

    /// Note the `rawValue`s for `name` and `foundingYear` cases
    /// and that `vantagePoints` is not present
}

let cahokiaMounds = Landmark(
    name: "Cahokia Mounds",
    foundingYear: 600,
    location: "St. Louis",
    vantagePoints: [
        CLLocation(latitude: 38.6551, longitude: 90.0618)
    ]
)
let data = try JSONEncoder().encode(cahokiaMounds)

/**
 Contents of `data: Data` are as follows

 {
    "title": "Cahokia Mounds",
    "founding_date": 600,
    "location": "St. Louis"
 }
 */
```

# Additional Reference

- Encoding and Decoding Custom Types

- Using JSON with Custom Types